GALILEO Group 31 october 2025 Deadline Report

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

11 novembre 2025

Indice

1 Introduction **2**

2 Baseline NL **2**
2.1 General Procedure 2
2.2 Dataset Categorization and Contextualization 3
2.3 Architecture: 3
2.4 Results and Evaluation 4

3 Baseline Palimpzest **5**
3.1 Methodological Foundations 5
3.2 Retrieval Strategy: Generic and Unfiltered Corpus 5
3.3 Results and Evaluation 6

4 Baseline SQL **6**
4.1 General Procedure 6
4.2 Architecture and Implementation 7
4.3 Command-line Interface and Logical Flow 8

1 Introduction

In the previous project task we assessed the query execution on the database for each dataset and after that, we use galois_eval.py for the evaluation process on the results we obtained from our implementation. Now in this task we will describe the development process implemented for replicate the following baselines using the Internal Knowledge datasets:

- **SQL baseline**: The system prompt the LLM model directly with a SQL query.
- **NL baseline**: The system prompt the LLM model directly with a NL prompt.
- **Palimpzest baseline**: The system evaluates the pure reasoning and synthesis capabilities of the LLM when operating under a Retrieval-Augmented Generation (RAG) paradigm (**BONUS**).

For all these baselines we will show also the tables that reports the evaluation measures and metrics. After this brief introduction, let's take a look at all the details of the various baselines.

2 Baseline NL

The Baseline NL module provides a reference framework for evaluating Large Language Models (LLMs) through direct interaction via natural language prompts. This approach assesses the model's intrinsic reasoning and knowledge by prompting it with natural language (NL) questions, optionally enriched with structural and contextual information derived from a database schema or a specific query results. All outputs are standardized in a unified FULL JSON format to facilitate downstream evaluation.

2.1 General Procedure

1. **Prompt Loading:** Natural language prompts are loaded from dataset-specific resources (**queries_<dataset-name>.txt**). Each prompt corresponds to a query from the same dataset.
2. **LLM interaction:** Prompts are processed through a **LangChain-based pipeline**, which integrates:
   - The general task definition and LLM role specification.
   - The corresponding database schema, extracted via DuckDB.
   - Optional contextual data (e.g. query results for Model Context datasets).
   - The natural language question itself
3. **Response Parsing and Storage:** The model's output is parsed using a **PyddanticOutputParser**, ensuring compliance with the required FULL JSON format. Each response is stored in an individual JSON file, forming a consistent source for evaluation.

2.2 Dataset Categorization and Contextualization

1. **Internal knowledge:** In these datasets, the model relies exclusively on its pretrained knowledge and linguistic reasoning capabilities. The prompt includes only the natural language question and, optionally, the database schema, but no factual data retrieved from the database.

2. **Model Context:** These datasets implement a **Retrieval-Augmented Generation (RAG)-inspired** mechanism. Prior to querying the LLM, the corresponding SQL query is executed against the database, and the retrieved tuples are supplied to the model as external contextual input. This design allows the model to reason jointly over its internal representations and the retrieved data, thus improving factual consistency and grounding. Each resulting file produced by the module fits the following FULL JSON format:

```
{
    "result_set": [
        { "originaltitle": "The Three Musketeers" },
        { "originaltitle": "The Count of Monte Cristo" }
    ],
    "time": 1.84,
    "tokens": 312
}
```

where:

- **result_set:** Denotes the structured LLM output.
- **time:** Represents the total time taken by the LLM to generate the response.
- **tokens:** Indicates the number of tokens processed during the interaction

2.3 Architecture:

The architecture is composed of three main components:

1. **Configuration Layer:**
    - API keys for the supported providers (**IBMWatsonx** and **Google Gemini**) are loaded form a **.env** file.
    - The **Config_Loader** selects the target LLM provider, and the appropriate wrapper is instantiated through **get_llm_wrapper()**.
    - Each wrapper extends the **LLMBaseWrapper** class, implementing provider-specific logic for model initialization, token counting, and response handling.

2. **Chain Construction:**

    The **build_lcel_chain()** method constructs a **LangChain Expression Language (LCEL)** pipeline comprising:

- **Database Context Retrieval: get_db_context()** extracts schema information and, for Model Context datasets, executes the original SQL query using DuckDB database instance of corresponding dataset.
- **Prompt Composition: ChatPromptTemplate** merges two layers, a **SYSTEM_PROMPT** defining the LLM's role and behavior, and a **HUMAN_PROMPT** containing the NL question and formatting directives.
- **Structured Output Parsing:** The **PydanticOutputParser** enforces adherence to the predefined FULL JSON schema.

3. **Execution Flow**

- For each dataset, SQL queries and corresponding NL prompts are loaded.
- The **(prompt, query)** pairs are processed through **chain.invoke()**.
- Responses are parsed, validated, and augmented with the metadata such as execution time and token usage.
- The resulting files are serialized via **save_baseline_to_json()** for evaluation purposes.

2.4 Results and Evaluation

The results obtained from the Baseline NL module were evaluated using standard metrics to assess the performance of the LLMs in both Natural Language (NL) and SQL contexts. The model used for the **gemini** provider is: **gemini-2.5-flash**, instead for **IBMWatsonx** provider we used **meta-llama/llama-3-3-70b-instruct**. All systems are tested on the datasets described in Table 2 of the previous project task, excluding Premier and Fortune like the galois authors did.

Metric	**NL**	**SQL**
F1-CELL	0.391	0.537
CARDINAITY	0.644	0.733
TUPLE-CONSTR.	0.182	0.331
AVG-SCORE	0.406	0.534
#TOKENS(M)	39965	126946
AVG-TIME	4.812	7.561

Tabella 1: GEMINI results

Metric	**NL**	**SQL**
F1-CELL	0.443	0.338
CARDINAITY	0.693	0.577
TUPLE-CONSTR.	0.209	0.121
AVG-SCORE	0.448	0.345
#TOKENS(M)	14808	9090
AVG-TIME	4.691	1.192

Tabella 2: WATSONX results

3 Baseline Palimpzest

During the development of the NL baseline, we initially implemented a rudimentary form of RAG-style retrieval, where query results were passed to the LLM within a specific natural language prompt. However, after reviewing the GALOIS paper and the methodology outlined on the Palimpsest project website (https://palimpzest.org/), it became evident that our implementation did not fully align with the principles and objectives described in those sources. Consequently, we designed a more conceptually consistent framework tailored to the goals of the Palimpsest approach, as detailed above.

3.1 Methodological Foundations

The Palimpsest Baseline strictly evaluates the pure reasoning and synthesis capabilities of the Large Language Model (specifically the Watsonx service utilizing **meta-llama/llama-3-3-70b-instruct** in our case) when operating under a Retrieval-Augmented Generation (RAG) paradigm. The core objective is to ensure the model responds solely based on the explicitly provided context, thereby isolating its performance from its internal pre-trained knowledge (Zero-Shot Knowledge).

The constrained input provided to the LLM consists of:

- **Database Schema**: The explicit definition of tables and columns
- **RAG Corpus** (**raw_data**): A textual block representing a subset of records extracted from the database.
- **NL query**: The question posed by the user.

The model is strictly instructed to produce the output in a validated FULL JSON format, utilizing only the information contained within the RAG Corpus.

3.2 Retrieval Strategy: Generic and Unfiltered Corpus

A crucial aspect of this baseline implementation is the deliberate choice of a generic and non-optimized retrieval strategy for generating the RAG Corpus.

The query used to extract the contextual data is standardized as: **SELECT * FROM <table_name> LIMIT 200;**

The use of this query is essential for the methodology and serves a dual purpose in testing the LLM's capabilities:

1. **Testing Internal Filtering (Pure Reasoning)**: By providing a large, unsorted, and unfiltered block of data (i.e., the first 200 records, which are often less relevant to the specific query), the experiment forces the LLM to perform internal logical filtering and data extraction within the provided text context. This measures the model's ability to execute complex operations on a noisy and voluminous corpus, moving beyond simple keyword matching.
2. **Validating Faithfulness (Hallucination Control)**: If the data required to answer a specific query is not present within the 200 extracted rows, the LLM must respond with an empty set ("result_set": []). This constraint validates the model's faithfulness to the input context and its resistance to hallucinating information from its pretrained knowledge base. The output on the PREMIER dataset (Queries 3, 4) confirming these empty results validated this methodological approach.

3. **Technical Constraints and Parameter Adjustment (LIMIT 200)**: The decision to cap the RAG Corpus at 200 rows was primarily driven by the strict **technical limitations** of the target LLM environment:

    - **Token Context Limit**: The LLama 3 model has a maximum context window of 131,072 tokens. Initial testing with higher limits ($L = 400$ or $L = 500$) resulted in API Status 400 (Bad Request) errors, as the combined size of the schema, prompt, and RAG Corpus exceeded the model's capacity. $L = 200$ was established as the maximum stable limit.
    - **Output Parsing Degradation**: Contexts exceeding $L = 200$ often led to the LLM ignoring the strict formatting instructions. The model would generate conversational preamble text before the required JSON object, causing Pydantic parsing failures. The limit of $L = 200$ proved to be the optimal threshold for maintaining high adherence to the required pure JSON output format.
    - **Acceptable Measurement**: Although LIMIT 200 does not cover the entire dataset of fortune for instance, it is sufficient to test the LLM's reasoning ability on the most populated subset of the database, accepting that accuracy metrics will reflect the simulated RAG Corpus limitation.

Finally, this implementation of the PZ baseline effectively provides metrics for both the faithfulness to context and the precision of reasoning of the LLM within a constrained and technically demanding environment.
In addition as proof of the principles described above, in the dataset Premier some results are empty when the relevant data was not present in the retrieved rows of database. In other hand when tha data are present in the retrieved rows, the results are not empty.

3.3 Results and Evaluation

Metric	NL	SQL
AVG-SCORE	0.393	X
#TOKENS(M)	2390	X

Tabella 3: WATSONX results

4 Baseline SQL

The Baseline SQL module extends the evaluation framework to the case where the model is queried through SQL statements instead of natural language questions. The goal is to analyse how the different LLM providers are able to interpret SQL queries, reason over the underlying database schema and data, and produce a structured JSON answer that can be evaluated with the same metrics used for the NL baseline.

4.1 General Procedure

The Baseline SQL pipeline follows a procedure that is conceptually similar to the one adopted for the NL baseline:

1. **Query loading.** For each dataset all SQL statements are loaded from the corresponding `queries_<dataset>.sql` file by means of the helper function `load_queries_from_folde`

2. **Schema and context extraction.** The DuckDB database associated with the dataset is opened and its schema is extracted. As for the NL baseline, datasets are divided into two categories:

   - *Internal Knowledge (IK)*: only the database schema is provided to the model as external context;
   - *Model Context (MC)*: the SQL query is also executed on DuckDB and the tuples returned by the query are supplied to the model as additional context.

3. **Prompt construction.** A structured prompt is built that combines a high level description of the task (SQL interpretation and result formatting), the schema information and, for MC datasets, a textual representation of the rows returned by the query, together with the SQL statement itself. The helper `build_sql_prompt()` wraps the query into a concise instruction.

4. **LLM invocation and parsing.** The prompt is sent to the selected LLM provider (Google Gemini or IBM watsonx). The answer is parsed through a `PydanticOutputParser` that enforces a common FULL JSON schema including the fields `result_set`, `time` and `tokens`. In case of parsing errors, the raw content is logged and an empty `result_set` is returned together with timing information.

5. **Result serialisation.** For each query a JSON file is produced and saved in a provider–specific directory inside the results folder. These files are then used by the evaluation scripts to compute the same metrics already introduced for the NL baseline.

4.2 Architecture and Implementation

The implementation of the Baseline SQL module is split into two main components, one for each provider.

For **Google Gemini** the core logic is implemented in the `baseline_sql_gemini.py` module. The main responsibilities of this component are:

- opening the DuckDB database of the selected dataset and extracting the schema;
- executing, in the MC case, the SQL query to obtain the raw tuples;
- building the prompt by combining schema, optional raw data and the SQL statement;
- invoking the model and parsing the output with a Pydantic schema (`Response`) that contains the `result_set` and metadata such as execution time and token usage;
- writing one JSON file per query in a deterministic path that depends on the provider and the dataset.

The function `run_sql_baseline_gemini()` orchestrates these steps for one or more datasets. If no datasets are specified on the command line, the selection is read from the YAML configuration (field `database.run`).

For **IBM watsonx** the Baseline SQL follows the same high level structure. The helper `run_sql_baseline_watsonx()` loads the queries, distinguishes between IK and MC datasets, executes the appropriate baseline function and finally serialises the results using the same FULL JSON format. In this way the two providers can be compared directly on the same evaluation metrics.

4.3 Command-line Interface and Logical Flow

The unified command-line interface is implemented in `src/main.py` and controls the execution of both NL and SQL baselines. The script exposes:

- a positional argument `datasets`, that can contain zero, one or multiple dataset names (for example `WORLD`, `GEO`, `MOVIES`);
- the option `-mode`, with possible values `nl`, `sql` or `both`, which selects whether to run only the NL baseline, only the SQL baseline or both;
- the option `-provider`, with possible values `gemini` and `watsonx`, that can override the provider defined in the YAML configuration file.

At a high level, the logical flow is the following:

1. parse the command-line arguments;
2. determine the list of datasets, either from the CLI or from the configuration;
3. according to the selected mode, invoke the NL baseline, the SQL baseline, or both;
4. for each baseline, dispatch to the provider–specific implementation (Gemini or watsonx) and store the resulting JSON files.

The quantitative results of the Baseline SQL module are reported together with those of the NL baseline in the tables that summarise, for each provider, the F1–CELL, CARDINALITY and TUPLE–CONSTRAINT metrics, their average score, the average inference time and the number of tokens produced.